CENG495 - HW2 | BATUHAN AKÇAN

1. I have chosen the project microservice-kubernetes. The reason why I chose this is it contains microservices built with Spring Boot, which I like the most among backend frameworks since it is used in large-scale applications. Another reason is it includes “kubernetes” in its name, so maybe it will be easier to integrate with kubernetes.

2. I have downloaded minikube. I tried to start it with the following command: “minikube start”. But it gave me an error:

   Exiting due to PROVIDER_DOCKER_VERSION_EXIT_1: "docker version --format <no value>-<no value>:<no value>" exit status 1: error during connect: Get "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/v1.47/version" : open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.

   I figured out that I should have started Docker Desktop in order for minikube to be able to start. Docker has already been installed on my machine, so no need to install it. After starting, the command was working.

3.
   a. I have downloaded the skaffold executable which was named “skaffold-windows-amd64.exe”. When I added it to the system PATH variable, writing “skaffold” to the terminal was not working. I figured out that I should have renamed the executable as “skaffold.exe”. After renaming, “skaffold” command was working.

   b. Then I have cloned the repository of the project I have chosen and ran “scaffold init” in it. This command generated a “skaffold.yaml” file.

   c. I ran the commands “minikube start --profile custom”, “skaffold config set --global local-cluster true”, “minikube -p custom docker-env” respectively, in the project directory, as stated in the skaffold quickstart tutorial. The last command has shown me some details:
   $Env:DOCKER_TLS_VERIFY = "1"

$Env:DOCKER_HOST = "tcp://127.0.0.1:53189"
$Env:DOCKER_CERT_PATH = "C:\Users\batuh\.minikube\certs"
$Env:MINIKUBE_ACTIVE_DOCKERD = "custom"

4.

a. When I ran “skaffold dev”, I got the error:

getting hash for artifact "docker.io/ewolff/microservice-kubernetes-demo-catalog": getting dependencies for "docker.io/ewolff/microservice-kubernetes-demo-catalog": file pattern [target/microservice-kubernetes-demo-catalog-0.0.1-SNAPSHOT.jar] must match at least one file

As far as I understood, this error implied that there existed no .jar file. So I created one for each microservice using the “mvn clean package” command. After that, “skaffold dev” command worked.

b. “kubectl get deployments”:

```
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
apache     1/1     1            1           17m
catalog    1/1     1            1           17m
customer   1/1     1            1           17m
order      1/1     1            1           17m
```

“kubectl get pods”:

```
NAME                        READY   STATUS    RESTARTS   AGE
apache-6995c5b99-kz529      1/1     Running   0          17m
catalog-86c6dfbb49-j2xmp    1/1     Running   0          17m
customer-5946b5dbd9-4ctsf   1/1     Running   0          17m
order-6b5bfcd477-g9jlg      1/1     Running   0          17m
```

“kubectl get services”:

```
NAME         TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
apache       LoadBalancer   10.108.29.28    <pending>     80:30808/TCP     18m
catalog      LoadBalancer   10.99.164.207   <pending>     8080:32682/TCP   18m
customer     LoadBalancer   10.104.157.83   <pending>     8080:30869/TCP   18m
kubernetes   ClusterIP      10.96.0.1       <none>        443/TCP          46h
order        LoadBalancer   10.103.56.120   <pending>     8080:32077/TCP   18m
```

5. I have picked the smollm2:135m-instruct-q4_K_M model because it is a small model that can be run without too much resource. Then I have created a sub-directory in the project and a Dockerfile inside it. The Dockerfile content is:

```
FROM ollama/ollama:latest

# Set environment variables
ENV MODEL_NAME=smollm2:135m-instruct-q4_K_M

# Expose the model server port
EXPOSE 8080

# Run the model server
RUN ollama serve & sleep 5 && ollama run
smollm2:135m-instruct-q4_K_M
```

After that I ran the commands “docker build -t docker.io/bakcan51/ollama-model:latest .” and “docker push docker.io/bakcan51/ollama-model:latest”, respectively.

Then I have added to the pre-existing “skaffold.yaml” file the following:

```
- image: docker.io/bakcan51/ollama-model
      context: ollama-model
      docker:
        dockerfile: Dockerfile
```

I have added to the pre-existing “microservices.yaml” file the following:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    run: ollama-model
  name: ollama-model
spec:
  replicas: 1
```

```
  selector:
    matchLabels:
      run: ollama-model
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        run: ollama-model
    spec:
      containers:
      - image: docker.io/bakcan51/ollama-model:latest
        name: ollama-container
        ports:
        - containerPort: 8080
        env:
        - name: MODEL_NAME
          value: smollm2:135m-instruct-q4_K_M
        resources:
          limits:
            memory: 4Gi
            cpu: 2000m
          requests:
            memory: 2Gi
            cpu: 1000m
status: {}
```

```
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    run: ollama-model
  name: ollama-model
spec:
  ports:
```

```
  - port: 8080
    protocol: TCP
    targetPort: 8080
  selector:
    run: ollama-model
  type: LoadBalancer
status:
  loadBalancer: {}
```

And then ran the command “kubectl apply -f microservices.yaml”. Then I ran “skaffold dev” again.

6. After I ran the command “minikube service apache --url”, the frontend was exposed. I was able to reach the frontend with the url “http://127.0.0.1:57492/”. I observed that, every time I re-run this command, the port changes.

   “kubectl get deployments”:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
apache	1/1	1	1	2m11s
catalog	1/1	1	1	2m11s
customer	1/1	1	1	2m11s
ollama-model	1/1	1	1	2m11s
order	1/1	1	1	2m11s

   “kubectl get pods”:

NAME	READY	STATUS	RESTARTS	AGE
apache-7c4b98d45c-r7tmn	1/1	Running	0	2m33s
catalog-58d69dfc7b-nkj7s	1/1	Running	0	2m33s
customer-55d9d45676-xf8tf	1/1	Running	0	2m33s
ollama-model-5b4bf7d4f-k4nld	1/1	Running	0	2m33s
order-5db75c89db-j4svh	1/1	Running	0	2m33s

   “kubectl get services”:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
apache	LoadBalancer	10.103.84.76	<pending>	80:32692/TCP	2m46s
catalog	LoadBalancer	10.96.14.164	<pending>	8080:30466/TCP	2m46s
customer	LoadBalancer	10.97.16.234	<pending>	8080:30686/TCP	2m46s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	4d7h
ollama-model	NodePort	10.97.139.250	<none>	8080:30081/TCP	2m46s
order	LoadBalancer	10.105.169.8	<pending>	8080:30687/TCP	2m46s

“kubectl get endpoints”:

```
NAME           ENDPOINTS          AGE
apache         10.244.0.210:80    3m11s
catalog        10.244.0.213:8080  3m11s
customer       10.244.0.211:8080  3m11s
kubernetes     192.168.49.2:8443  4d7h
ollama-model   10.244.0.214:8080  3m11s
order          10.244.0.212:8080  3m11s
```

7. Firstly, I have updated “microservices.yaml” file and changed the ollama-model service configuration to this:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: ollama-model
  labels:
    run: ollama-model
spec:
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30081  # Set to a fixed port
  selector:
    run: ollama-model
```

Then I have updated the “index.html” which resides in the “apache” folder. This is the updated version:

```html
<!DOCTYPE html>
<html>
<head>
<title>Order Processing</title>
<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.2.0/
css/bootstrap.min.css" />
```

```
<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.2.0/
css/bootstrap-theme.min.css" />
<script
src="https://maxcdn.bootstrapcdn.com/bootstrap/3.2.0/j
s/bootstrap.min.js"></script>
</head>
<body>
    <h1>Order Processing</h1>
    <div class="container">
        <div class="row">
            <div class="col-md-4">
                <a
href="/customer/list.html">Customer</a>
            </div>
            <div class="col-md-4">List / add / remove
customers</div>
        </div>
        <div class="row">
            <div class="col-md-4">
                <a
href="/catalog/list.html">Catalog</a>
            </div>
            <div class="col-md-4">List / add / remove
items</div>
        </div>
        <div class="row">
            <div class="col-md-4">
                <a
href="/catalog/searchForm.html">Catalog</a>
            </div>
            <div class="col-md-4">Search Items</div>
        </div>
        <div class="row">
            <div class="col-md-4">
```

```
                <a href="/order/">Order</a>
            </div>
            <div class="col-md-4">Create an
order</div>
        </div>
        <div class="row">
            <div class="col-md-12">
                <h2>Chat with the AI</h2>
                <div id="chat-window" style="height:
300px; border: 1px solid #ccc; overflow-y: scroll;
margin-bottom: 10px;"></div>
                <input type="text" id="chat-input"
placeholder="Type your message..."
class="form-control" />
                <button onclick="sendMessage()"
class="btn btn-primary" style="margin-top:
10px;">Send</button>
            </div>
        </div>
    </div>

    <script>
    async function sendMessage() {
        const inputElement =
document.getElementById("chat-input");
        const chatWindow =
document.getElementById("chat-window");
        const message = inputElement.value.trim();

        if (!message) return;

        chatWindow.innerHTML += `<div
class="user-message"><strong>You:</strong>
${message}</div>`;
```

```
        try {
            const response = await
fetch("http://192.168.49.2:30081
/api/chat", {
                method: "POST",
                headers: { "Content-Type":
"application/json" },
                body: JSON.stringify({
                    model:
"smollm2:135m-instruct-q4_K_M",
                    prompt: message
                })
            });

            const data = await response.json();
            chatWindow.innerHTML += `<div
class="bot-message"><strong>Bot:</strong>
${data.response}</div>`;
        } catch (error) {
            chatWindow.innerHTML += `<div
class="error-message"><strong>Error:</strong>
${error.message}</div>`;
        }

        inputElement.value = "";
        chatWindow.scrollTop =
chatWindow.scrollHeight;
    }
    </script>
</body>
</html>
```

I ran the commands “skaffold build”, “skaffold dev” in a terminal, “minikube service --all” in another terminal and then the web application opened.